LAPORAN PRAKTIKUM

Struktur Data

MODUL 4

SINGLY LINKED LIST (BAGIAN PERTAMA)



Disusun oleh:

Elsa Dwi Rizqiyanti

109082500090

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

Guided

Link Repo Github Modul 4 : Struktur-Data/Modul4 at main · elsadwiyt/Struktur-Data

Guided 1

Source Code

```
#include <iostream>

using namespace std;

typedef int infotype;
typedef struct elmlist *address;

struct elmlist {
    infotype info;
    address next;
};

struct List {
    address first;
};

void createList(List &L) {
    L.first = NULL;
}

address alokasi(infotype x) {
    address P = new elmlist;
    P->info = x;
    P->next = NULL;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = NULL;
}

void insertFirst(List &L, address P) {
    P->next = L.first;
    L.first = P;
}

void insertLast(List &L, address P) {
    if (L.first == NULL) {
```

```
            insertFirst(L, P);
        } else {
            address Q = L.first;
            while (Q->next != NULL) {
                Q = Q->next;
            }
            Q->next = P;
        }
    }

    void insertAfter(List &L, address P, address Prec) {
        if (Prec != NULL) {
            P->next = Prec->next;
            Prec->next = P;
        }
    }

    void deleteFirst(List &L, address &P) {
        if (L.first != NULL) {
            P = L.first;
            L.first = P->next;
            P->next = NULL;
        }
    }

    void deleteLast(List &L, address &P) {
        if (L.first != NULL) {
            if (L.first->next == NULL) {
                deleteFirst(L, P);
            } else {
                address Q = L.first;
                while (Q->next->next != NULL) {
                    Q = Q->next;
                }
                P = Q->next;
                Q->next = NULL;
            }
        }
    }

    void deleteAfter(List &L, address &P, address Prec) {
        if (Prec != NULL && Prec->next != NULL) {
            P = Prec->next;
            Prec->next = P->next;
            P->next = NULL;
        }
    }
```

```
void printInfo(List L) {
    address P = L.first;
    while (P != NULL) {
        cout << P->info << ' ';
        P = P->next;
    }
    cout << '\n';
}

int main() {
    List L;
    address P1, P2, P3, P4, P5;

    createList(L);

    P1 = alokasi(2);
    insertFirst(L, P1);

    P2 = alokasi(0);
    insertFirst(L, P2);

    P3 = alokasi(8);
    insertFirst(L, P3);

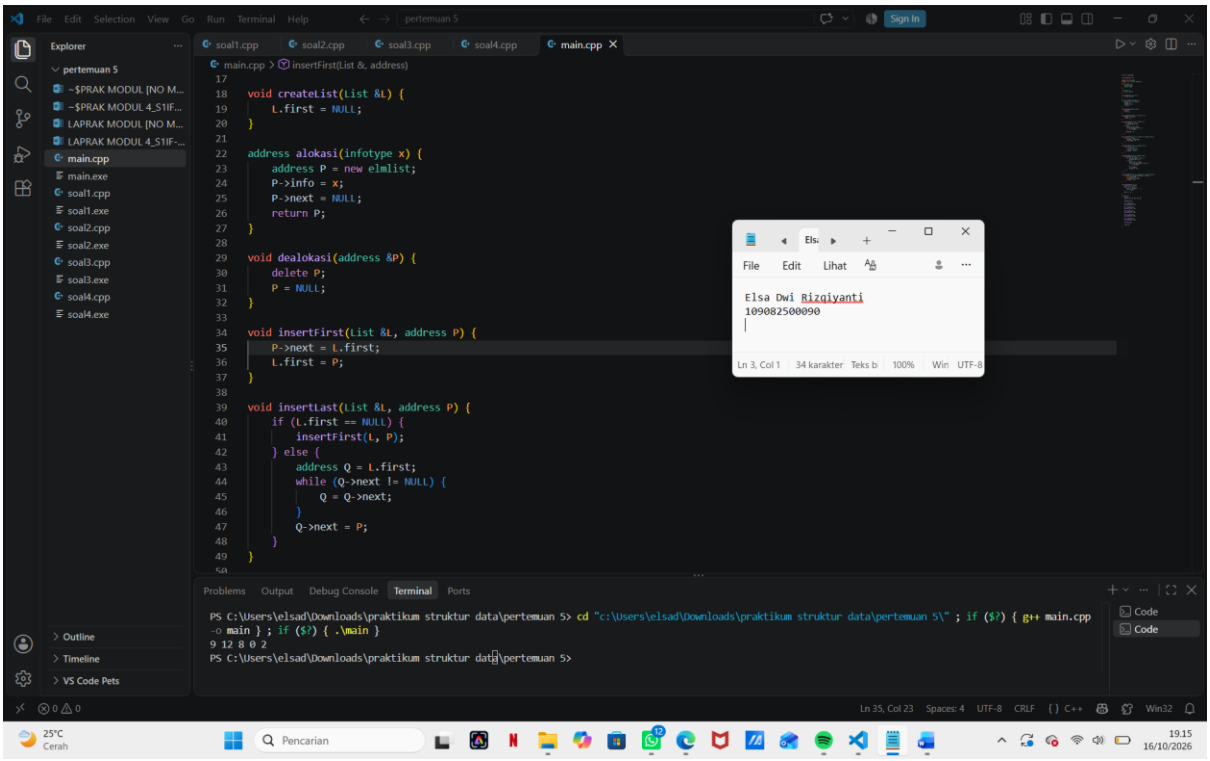
    P4 = alokasi(12);
    insertFirst(L, P4);

    P5 = alokasi(9);
    insertFirst(L, P5);

    printInfo(L);

    return 0;
}
```

Screenshoot program



Deskripsi program

Program ini adalah implementasi struktur data *Single Linked List* (senarai berantai tunggal) dalam bahasa C++ yang dirancang untuk menyimpan sekumpulan data integer secara dinamis menggunakan *node* yang saling terhubung. Program menyediakan serangkaian fungsi fundamental untuk memanipulasi data, meliputi manajemen memori (alokasi dan dealokasi), penambahan data (di awal, akhir, maupun di tengah list), penghapusan data, serta fungsi penelusuran untuk mencetak isi list ke layar. Pada fungsi utamanya, program mendemonstrasikan penggunaan fungsi insertFirst dengan memasukkan lima buah bilangan (2, 0, 8, 12, dan 9) secara berurutan ke bagian paling depan list, sehingga saat keseluruhan isi list dicetak, data ditampilkan dalam urutan terbalik dari proses pemasukkannya (9 12 8 0 2) layaknya prinsip tumpukan (LIFO).

LATIHAN SOAL MODUL

1. **Soal 1**

Source Code

```
#include <iostream>
#include <string>

using namespace std;

typedef string infotype;
typedef struct elmlist *address;

struct elmlist {
    infotype info;
    address next;
};

struct List {
    address first;
};

// Primitif Dasar ADT
void createList(List &L) {
    L.first = nullptr;
}

address alokasi(infotype x) {
    address P = new elmlist;
    P->info = x;
    P->next = nullptr;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = nullptr;
}

bool isEmpty(List L) {
    return L.first == nullptr;
}

void insertLast(List &L, address P) {
    if (isEmpty(L)) {
        L.first = P;
    } else {
        address temp = L.first;
        while (temp->next != nullptr) {
```

```
            temp = temp->next;
        }
        temp->next = P;
    }
}

void deleteFirst(List &L, address &P) {
    if (!isEmpty(L)) {
        P = L.first;
        L.first = P->next;
        P->next = nullptr;
    }
}

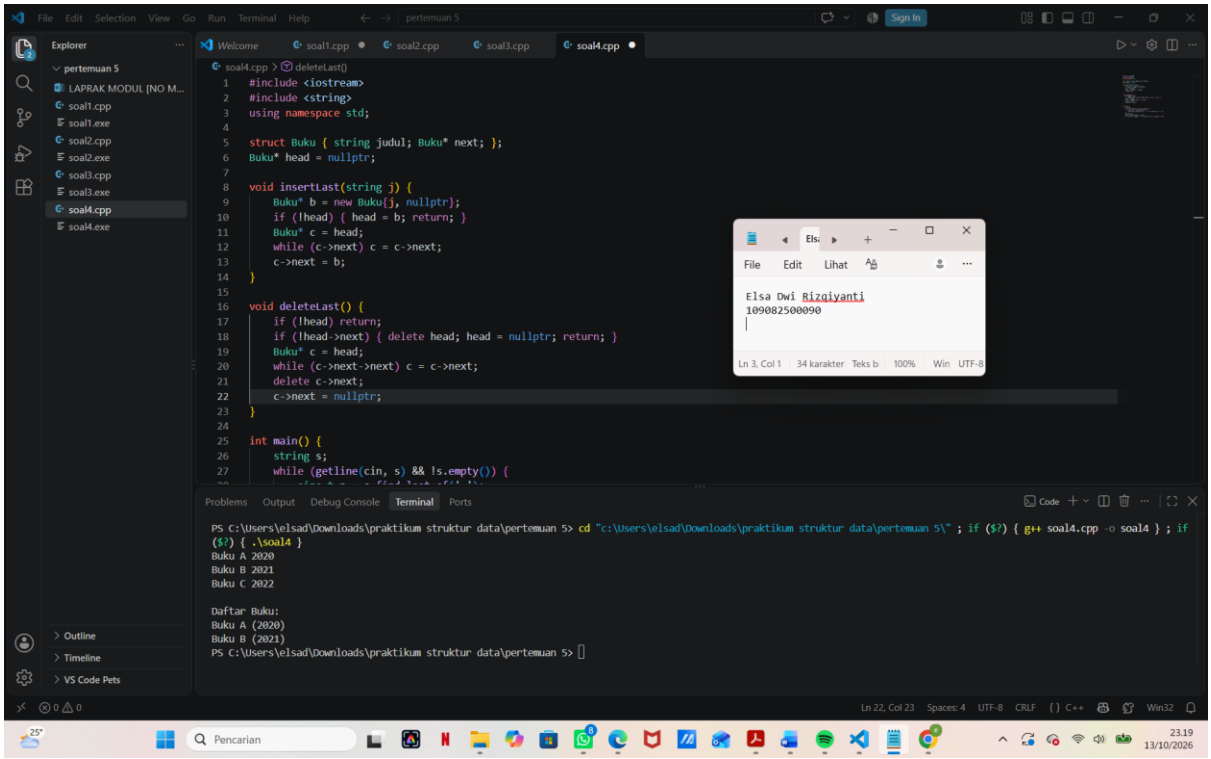
void printInfo(List L) {
    if (isEmpty(L)) {
        cout << "Antrean kosong" << endl;
    } else {
        cout << "Antrean: ";
        address temp = L.first;
        while (temp != nullptr) {
            cout << temp->info;
            if (temp->next != nullptr) cout << " -> ";
            temp = temp->next;
        }
        cout << endl;
    }
}

int main() {
    List L;
    createList(L);
    int pilihan;
    string nama;

    while (cin >> pilihan && pilihan != 3) {
        if (pilihan == 1) {
            cin >> nama;
            address P = alokasi(nama);
            insertLast(L, P);
            printInfo(L);
        } else if (pilihan == 2) {
            address P = nullptr;
            deleteFirst(L, P);
            if (P != nullptr) {
                cout << "Melayani: " << P->info << endl;
                dealokasi(P);
            }
```

```
                printInfo(L);
            }
        }
        return 0;
    }
```

Screenshoot program



Deskripsi program

Program C++ ini mengimplementasikan struktur data *Singly Linked List* untuk mensimulasikan sistem antrean nasabah bank secara dinamis menggunakan pointer. Di dalamnya terdapat pembentukan ADT dasar seperti pembuatan list kosong, alokasi memori *node*, dealokasi, serta pengecekan kondisi kosong. Pengguna dapat mengoperasikan program melalui menu interaktif untuk menambahkan nasabah baru ke akhir antrean (*insertLast*), melayani dan menghapus nasabah di urutan pertama (*deleteFirst* beserta dealokasi memorinya), serta memantau status antrean secara *real-time* hingga memilih keluar.

2. **Soal 2**

Source Code

```
#include <iostream>

using namespace std;

typedef int infotype;
typedef struct elmlist *address;

struct elmlist {
    infotype info;
    address next;
};

struct List {
    address first;
};

void createList(List &L) {
    L.first = nullptr;
}

address alokasi(infotype x) {
    address P = new elmlist;
    P->info = x;
    P->next = nullptr;
    return P;
}

bool isEmpty(List L) {
    return L.first == nullptr;
}

void insertLast(List &L, address P) {
    if (isEmpty(L)) {
        L.first = P;
    } else {
        address temp = L.first;
        while (temp->next != nullptr) {
            temp = temp->next;
        }
        temp->next = P;
    }
}

void insertAfter(List &L, address P, address Prec) {
    if (Prec != nullptr) {
        P->next = Prec->next;
```

```
            Prec->next = P;
        }
    }

    void printInfo(List L) {
        address temp = L.first;
        while (temp != nullptr) {
            cout << temp->info;
            if (temp->next != nullptr) cout << " -> ";
            temp = temp->next;
        }
        cout << endl;
    }

    int main() {
        int n, val, target, newData;
        List L;
        createList(L);

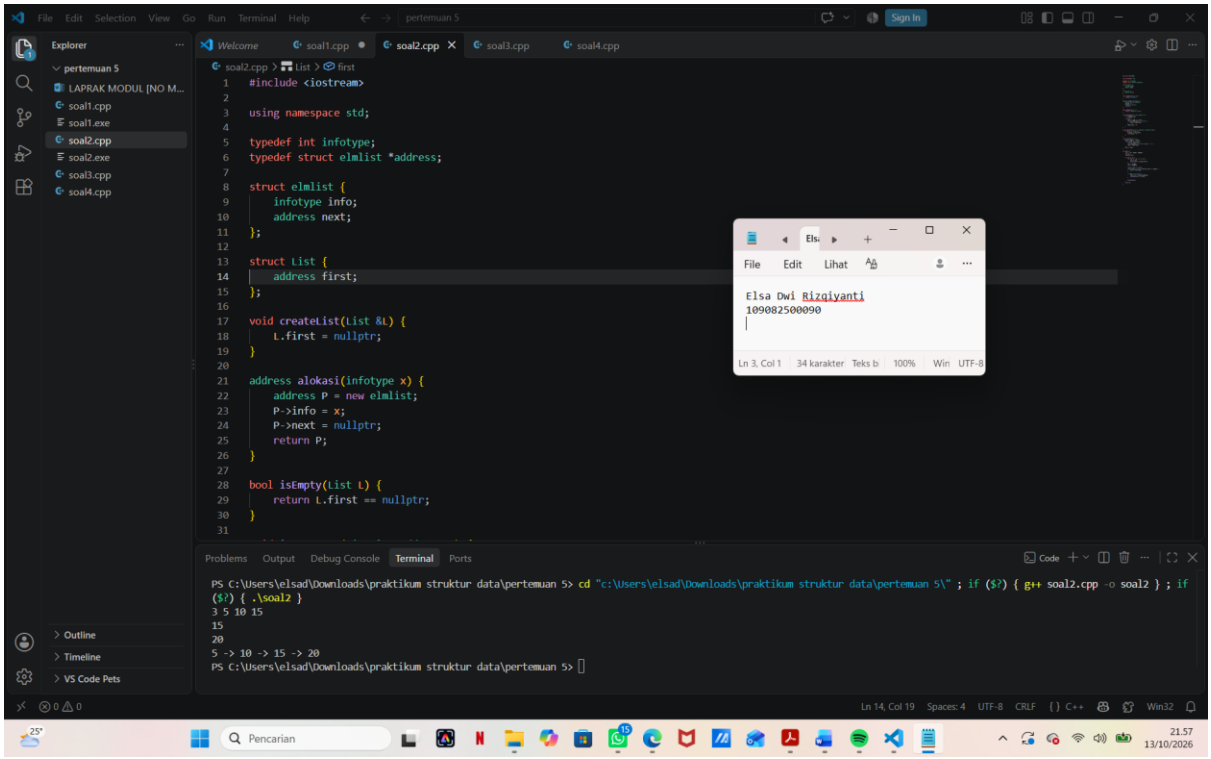
        if (cin >> n) {
            for (int i = 0; i < n; i++) {
                cin >> val;
                insertLast(L, alokasi(val));
            }
            cin >> target;
            cin >> newData;

            address Prec = L.first;
            while (Prec != nullptr && Prec->info != target) {
                Prec = Prec->next;
            }

            if (Prec != nullptr) {
                address P = alokasi(newData);
                insertAfter(L, P, Prec);
            }

            printInfo(L);
        }
        return 0;
    }
```

Screenshoot program



Deskripsi program

Program C++ ini mengimplementasikan struktur data *Singly Linked List* untuk melakukan pencarian dan penyisipan data baru tepat setelah elemen tertentu ditemukan (*insertAfter*). Program diawali dengan pembentukan list kosong dan alokasi memori dinamis untuk memasukkan sejumlah N elemen awal ke bagian akhir list (*insertLast*). Selanjutnya, program membaca elemen target yang dicari serta nilai data baru, menelusuri list hingga target ditemukan untuk disisipkan node baru di belakangnya, lalu menampilkan seluruh isi list secara terurut.

3. **Soal 3**

Source Code

```
#include <iostream>

using namespace std;

typedef int infotype;
typedef struct elmlist *address;

struct elmlist {
    infotype info;
    address next;
};

struct List {
    address first;
};

void createList(List &L) {
    L.first = nullptr;
}

address alokasi(infotype x) {
    address P = new elmlist;
    P->info = x;
    P->next = nullptr;
    return P;
}

bool isEmpty(List L) {
    return L.first == nullptr;
}

void insertLast(List &L, address P) {
    if (isEmpty(L)) {
        L.first = P;
    } else {
        address temp = L.first;
        while (temp->next != nullptr) {
            temp = temp->next;
        }
        temp->next = P;
    }
}

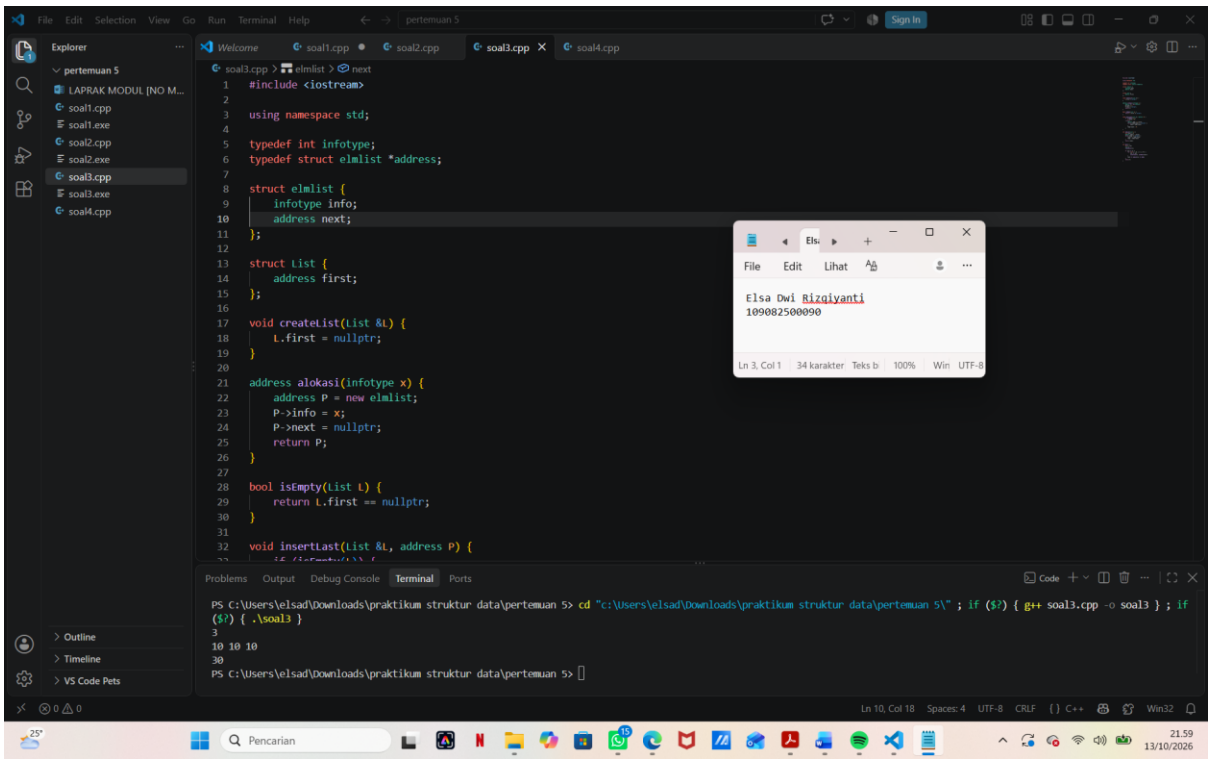
int sumList(List L) {
    int total = 0;
```

```
        address temp = L.first;
        while (temp != nullptr) {
            total += temp->info;
            temp = temp->next;
        }
        return total;
    }

    int main() {
        int n, val;
        List L;
        createList(L);

        if (cin >> n) {
            for (int i = 0; i < n; i++) {
                cin >> val;
                insertLast(L, alokasi(val));
            }
            cout << sumList(L) << endl;
        }
        return 0;
    }
```

Screenshoot program



Deskripsi program

Program C++ ini mengimplementasikan struktur data *Singly Linked List* untuk menghitung jumlah total (*sum*) dari seluruh elemen bilangan bulat yang dimasukkan ke dalam list. Program diawali dengan menginisialisasi list kosong, mengalokasikan memori secara dinamis, dan memasukkan N buah data angka ke akhir list menggunakan operasi *insertLast*. Setelah itu, fungsi sumList akan menelusuri setiap simpul pointer hingga akhir untuk mengakumulasikan seluruh nilai elemen dan mencetak hasil penjumlahannya ke layar.

4. **Soal 3**

Source Code

```
#include <iostream>
#include <string>
using namespace std;

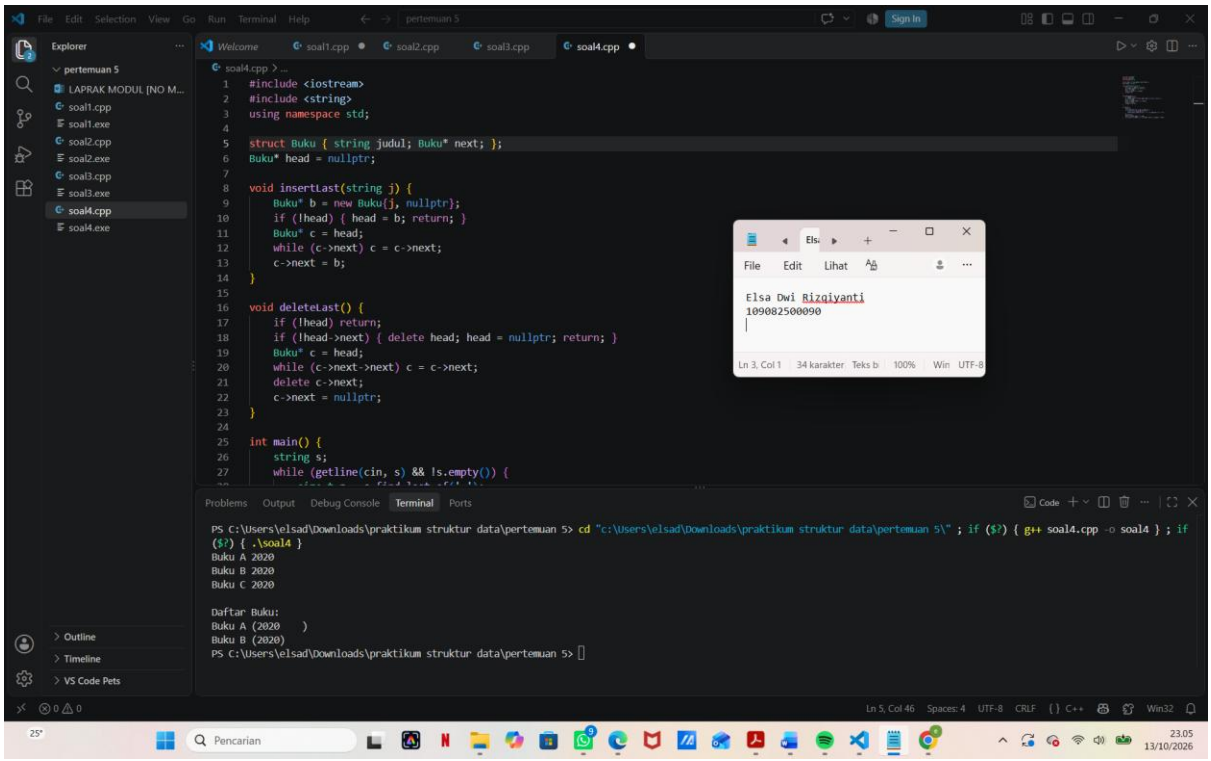
struct Buku { string judul; Buku* next; };
Buku* head = nullptr;

void insertLast(string j) {
    Buku* b = new Buku{j, nullptr};
    if (!head) { head = b; return; }
    Buku* c = head;
    while (c->next) c = c->next;
    c->next = b;
}

void deleteLast() {
    if (!head) return;
    if (!head->next) { delete head; head = nullptr; return; }
    Buku* c = head;
    while (c->next->next) c = c->next;
    delete c->next;
    c->next = nullptr;
}

int main() {
    string s;
    while (getline(cin, s) && !s.empty()) {
        size_t p = s.find_last_of(' ');
        insertLast(s.substr(0, p) + " (" + s.substr(p + 1) +
")");
    }
    deleteLast();
    cout << "Daftar Buku:" << endl;
    for (Buku* c = head; c; c = c->next) cout << c->judul <<
endl;
}
```

Screenshoot program



Deskripsi program

Program C++ tersebut adalah implementasi *linked list* tunggal untuk mengelola daftar buku yang membaca input teks berformat judul dan keterangan, menyimpannya melalui fungsi insertLast, menghapus elemen terakhir dengan fungsi deleteLast, lalu menampilkan sisa daftar buku ke layar.